图 1: R97 系统概览与保语义反混淆流水线

Figure 1: R97 System Architecture & Semantic-Preserving Deobfuscation Pipeline



代码转换与保护示例

[阶段 0] 原始混淆代码 (Input)

```
// 字符串数组加密 + 小程序分支
var _0x4a = ['request', 'undefined'];
var val = 1 + 2 + 3;
if (typeof wx !== _0x4a[1]) {
  wx[_0x4a[0]]({ url: '/pay' });
}
var time = Date.now();

待去混淆与常量折叠 ➔
```

反混淆

传统反混淆工具 (无 R97)

```
// 纯常量折叠成功:
var val = 6;
// ✶ 离线 Node 求值误判 typeof wx 为 "undefined"!
// 死代码消除将整个微信支付分支物理抹除 (Dead Code Pruning)!
var time = 1760000000000; // 动态时钟被死锁为假常量
```

✶ 目标真机运行时崩溃! (100% 致命破坏)

核心能力丢失, 真机执行报 TypeError / 白屏死锁!

R97 守门

集成 R97 守门保护器

```
// 1. 纯语言计算正常享受常量折叠优化:
var val = 6;
// 2. R97 两阶段接力: 字符串成功解密, 宿主敏感分支 100% 完整保留!
if (typeof wx !== "undefined") { wx.request({ url: '/pay' }); }
var time = Date.now(); // 动态状态公理拦截, 保留动态时钟
```

✅ 跨宿主部署 100% 正常运行! (0 崩溃, 0 误放行)

既享受了 39.44% 的常量优化效用, 又彻底捍卫了原程序语义保真!

设计原则 (Design Principles): [1] 三域生态覆盖 (Web/Node/小程序); [2] 需求与目标正交隔离 (Required ⊥ Target); [3] 契约元能力降维; [4] 模块闭包自底向上拓扑传递; [5] Double-Oracle 双重闭环验证; [6] 100% 守住零误放行安全底线。